

Distributed Systems Homework - 3

Section 1 | Section 2 | Section 3

Section 1: MapReduce

Q1. Distributed Matrix Multiplication using MapReduce (Row-Row Method)

Problem Statement

You are given two matrices, A (of dimension $m \times n$) and B (of dimension $n \times p$). Implement a MapReduce program that computes $C = A \times B$ using the Row-Row method. Each row of C is computed as a weighted sum of the rows of B, where the weights are the entries of the corresponding row of A:

$$c_i = a_{i,1} B_{1,:} + a_{i,2} B_{2,:} + \cdots + a_{i,n} B_{n,:}$$

Input Constraints

- Dimensions m , n , and p range from small (e.g., 3×3 , for manual verification) to large (e.g., 1000×1000 or more, for timing/scaling).
- Matrices need not be square; test skewed shapes (tall $m \gg n$, or wide $n \gg m$).
- Test both cases: m divisible by the number of Mappers, and m not divisible.
- Edge cases: $m = 1$ (single row), $n = 1$ (single column in A / single row in B).
- Entries are integers.

MapReduce Execution Flow Requirements

- **Data Distribution:** Matrix A is stored in a distributed file system and processed row-by-row. Matrix B must be fully replicated to the local memory of every Mapper node (e.g., using the framework's Distributed Cache).
- **Map Phase:** The `map()` function processes its assigned rows of matrix A independently, without inter-worker communication, utilizing the fully replicated matrix B. You must determine the appropriate intermediate key-value pairs to emit.
- **Reduce Phase:** The framework automatically shuffles and groups the emitted data. The `reduce()` function must process these intermediate values and write the final output to correctly reconstruct matrix C.

Example 1: Even Split (Assuming 3 Mappers)

$$A_{3 \times 2} = \begin{pmatrix} 1 & 2 \\ 0 & 3 \\ -1 & 4 \end{pmatrix} \quad B_{2 \times 3} = \begin{pmatrix} 2 & 3 & 4 \\ 1 & 0 & -1 \end{pmatrix}$$

Distribution: Mapper 1 $\rightarrow$ row 1, Mapper 2 $\rightarrow$ row 2, Mapper 3 $\rightarrow$ row 3.

Computation:

$$\begin{aligned}c_1 &= 1 \cdot (2, 3, 4) + 2 \cdot (1, 0, -1) = (4, 3, 2) \\c_2 &= 0 \cdot (2, 3, 4) + 3 \cdot (1, 0, -1) = (3, 0, -3) \\c_3 &= -1 \cdot (2, 3, 4) + 4 \cdot (1, 0, -1) = (2, -3, -8)\end{aligned}$$

Collection:

$$C_{3 \times 3} = \begin{pmatrix} 4 & 3 & 2 \\ 3 & 0 & -3 \\ 2 & -3 & -8 \end{pmatrix}$$

Example 2: Uneven Split (Assuming 3 Mappers)

$$A_{4 \times 2} = \begin{pmatrix} 1 & 0 \\ 2 & -1 \\ 0 & 3 \\ 1 & 1 \end{pmatrix} \quad B_{2 \times 2} = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}$$

Distribution: Mapper 1 $\rightarrow$ 2 rows, Mapper 2 $\rightarrow$ 1 row, Mapper 3 $\rightarrow$ 1 row.

Computation:

$$c_1 = (1, 2) \quad c_2 = (2, 3) \quad c_3 = (0, 3) \quad c_4 = (1, 3)$$

Collection:

$$C_{4 \times 2} = \begin{pmatrix} 1 & 2 \\ 2 & 3 \\ 0 & 3 \\ 1 & 3 \end{pmatrix}$$

Q2. Single Source Shortest Path (SSSP) using MapReduce

Question

Find the shortest distance from a single source node (Node 0) to all other nodes in a weighted directed graph using an iterative MapReduce algorithm.

Constraints

- Number of vertices (V): $2 \leq V \leq 10000$
- Edge weights (w): $1 \leq w \leq 1000$. (The graph is guaranteed to have non-negative weights).
- The source node is always Node 0. Initial distance to Node 0 is 0, and to all other nodes is ∞ (or a suitably large integer like 1000000).

Input Format

- First line: Two integers V and E — number of vertices and edges.
- Next E lines: Three integers u , v , and w representing a directed edge from vertex u to vertex v with weight w .

Output Format

- Print V lines. Each line containing two space-separated values: `node_id shortest_distance`.
- If a node is unreachable from the source, output `INF` as its distance.
- The output must be sorted by `node_id` in ascending order.

Sample Input

```
4 4
0 1 5
0 2 2
2 1 1
1 3 4
```

Sample Output

```
0 0
1 3
2 2
3 7
```

Q3. Triangle Counting in an Undirected Graph (MapReduce)

Problem Statement

Implement a distributed algorithm using the MapReduce programming model to count the number of triangles in a given undirected graph. The input is an undirected graph stored as an edge list in a distributed file system. The algorithm must use the MapReduce framework to automatically parallelize the computation and efficiently find the total global count of triangles.

MapReduce Execution Flow Requirements

- **Data Distribution:** The edge list of the graph is split and distributed across Mappers.
- **Map Phase:** Each Mapper processes its assigned chunk of edges and produces intermediate key-value pairs.
- **Reduce Phase:** The framework shuffles and groups the intermediate data, and the Reducer uses these values to compute the final global triangle count.

Note: Ensure no double-counting of triangles occurs.

Constraints

- Number of vertices (V): $3 \leq V \leq 10^5$
- Number of edges (E): $3 \leq E \leq 10^6$
- Vertices are 0-indexed.

Input Format

- First line: Two integers V and $E \rightarrow$ number of vertices and edges.
- Next E lines: Two integers u and v representing an undirected edge between vertex u and vertex v .

Output Format

- A single integer representing the total global count of triangles in the graph.

Sample Input

```
4 5
0 1
1 2
2 0
2 3
3 0
```

Sample Output

```
2
```

(Explanation: The two triangles are formed by vertices $\{0, 1, 2\}$ and $\{0, 2, 3\}$.)

Section 2: Real World Problem - Solve using both MR and gRPC

Assignment Structure

This homework continues the real-world data analytics work from Homework 2. In Homework 2, each team was assigned one of Q7 or Q8 from the Real-World Applications section. In this homework, each team will continue with the same problem assigned to them in Homework 2. The input format, analytics, output format, and semantics remain as defined in the corresponding Homework 2 question. The same problem will now be implemented using two different distributed-system paradigms:

1. Q1: Batch Data Analytics using Hadoop MapReduce
2. Q2: Real-Time Streaming Analytics using gRPC

Each team will be assigned one question. The intention of this homework is to move from implementing a distributed algorithm to understanding how the choice of distributed-system model affects the design, implementation, communication, scalability, and performance of a real-world application.

General Instructions

- Continue with the same real-world analytics problem (Q7 or Q8) assigned to your team in Homework 2.
- Refer to Homework 2 for the complete problem statement, input format, required analytics, output format, and correctness requirements.
- Implement all analytics required by the corresponding Homework 2 problem.
- You are expected to make reasonable design decisions based on the distributed framework being used. Clearly explain and justify these decisions in your README/report.
- Submit source code, instructions for building and running the system, correctness verification, and performance results.
- Do not print debugging information as part of the required output.

Q1. Batch Data Analytics using Hadoop MapReduce

Question

Continue the real-world data analytics problem assigned to your team in Homework 2, and implement it using the Hadoop MapReduce programming model. The complete dataset is available before processing begins. Hadoop must distribute the input and execute the computation using mapper and reducer tasks. The objective is to study how the same real-world analytics problem can be implemented using a batch-oriented MapReduce model and to compare its behavior with the MPI implementation developed in Homework 2.

Required Environment

The solution must use:

- Hadoop: Apache Hadoop 3.3.6

- Job Scheduler: YARN
- Language: C++
- Interface: Hadoop Streaming

The mapper and reducer programs must be standalone C++ executables communicating with Hadoop through standard input and standard output. Hadoop Streaming must be used for the MapReduce execution. You should determine how the analytics from your Homework 2 problem can be decomposed into mapper and reducer operations. Your design may use one or more MapReduce stages as appropriate.

Implementation and Analysis

Implement all analytics required by your assigned Homework 2 problem. You are expected to analyze the computation and determine an appropriate MapReduce design. Your README/report should explain, at an appropriate level of detail:

- How the input is processed by the mapper(s).
- What intermediate information is generated.
- How the reducer(s) combine the information.
- How the final required analytics are obtained.
- Any additional MapReduce stages or aggregation required.

There is no prescribed mapper/reducer decomposition. You should choose and justify a design that is appropriate for the problem.

MPI vs MapReduce

Compare your MapReduce implementation with the MPI implementation of the same problem from Homework 2. Use equivalent datasets and meaningful configurations for the comparison. Your analysis should consider both quantitative and qualitative aspects of the two approaches. For example, you may consider:

- Execution time and throughput.
- Scaling with input size.
- Scaling with the number of processes/tasks.
- Memory usage.
- Communication and data movement.
- Programming complexity and implementation effort.
- Flexibility and ease of modifying the computation.

The above list is intended as guidance rather than a fixed checklist. Identify the factors that are relevant to your problem and use your experiments to support your observations.

Dataset Generation and Benchmarking

Use reproducible datasets for benchmarking. You may use the dataset generator from Homework 2 or extend it as necessary for larger experiments. Evaluate the behavior of your implementation

over multiple input sizes and appropriate Hadoop configurations. Compare the results with your MPI implementation.

Present your observations using suitable tables and/or plots and explain the reasons for the trends you observe.

Q2. Real-Time Data Analytics using gRPC

Question

Continue the real-world data analytics problem assigned to your team in Homework 2, but now consider the data as a continuous stream rather than as a complete batch. Implement a distributed real-time analytics system using gRPC. A streaming client should read a pre-generated dataset and replay its records as if they were arriving from a live source. The system must process the incoming records continuously and maintain the analytics required by your assigned Homework 2 problem. The system must use multiple analytics workers. A CLI dashboard must be provided to display the current analytics while the stream is being processed.

System Design

The overall architecture is intentionally left open. You should design an appropriate system using gRPC and multiple workers. At a high level, the system should contain:

- A client that streams records using gRPC.
- A gRPC server/coordinator that receives the stream.
- Multiple workers responsible for processing the incoming data.
- A mechanism for maintaining and combining the analytics state.
- A CLI client/dashboard for querying and displaying the current analytics.

You are free to decide how records are distributed among workers, how worker state is maintained, and how global analytics are obtained. The system should support analytics queries while data is still being ingested.

Analytics

All analytics required by your assigned Homework 2 problem must be maintained by the streaming system. The final analytics obtained after the complete stream has been processed should match the result of a correct sequential implementation of the Homework 2 problem. The dashboard should provide a useful view of the current state of the analytics while the stream is running.

gRPC Interface

Design an appropriate `.proto` interface for your system. At minimum, the interface must support:

- Streaming ingestion of records.
- Queries for the current analytics.

The exact RPC structure, message definitions, and division of functionality are left to you. Your README/report should explain the choices made in your interface and how they support the requirements of the system.

Streaming and Concurrency

The client should replay the dataset as a stream rather than submitting the entire dataset as one batch. Provide a configurable mechanism for controlling the rate or granularity at which records are sent so that the behavior of the system can be studied. The system should allow analytics queries to be made while ingestion is in progress. Your implementation should correctly handle concurrent updates and queries. You should investigate appropriate choices for:

- Number of workers.
- Record distribution strategy.
- Streaming/message batch size.
- Query frequency or number of concurrent query clients.

The exact choices are left to you; justify the configurations used in your experiments.

CLI Dashboard

The dashboard must be CLI-based. A web interface is not required. It should display the analytics of your assigned problem in a clear manner and should allow the current state of the system to be observed while the stream is being processed. The exact dashboard layout is left to you.

Performance Evaluation

Evaluate the system under multiple configurations and identify the factors that affect its performance. At minimum, your evaluation should investigate the effect of the number of workers and the way records are streamed. Useful measurements may include:

- Streaming throughput.
- Query latency.
- End-to-end processing time.
- Effect of worker count.
- Effect of streaming/message granularity.
- Effect of concurrent queries.
- CPU and memory usage, where appropriate.

The above are intended as guidance. Choose measurements that help explain the behavior of your system. Present suitable plots/tables and provide a short analysis of the observed results.

Dataset Generation and Reproducibility

Use the datasets from Homework 2 as the starting point for correctness testing. For performance experiments, generate sufficiently large datasets to demonstrate the behavior of the distributed

systems. Dataset generation must be reproducible. Document the parameters, random seed (if applicable), dataset sizes, and any other assumptions needed to reproduce the experiments.

Required Submission

The submission should contain the components necessary to build, execute, verify, and evaluate your solution.

For Q1 – Hadoop MapReduce, submit:

- Complete Hadoop Streaming implementation.
- Dataset generator or reproducible dataset-generation procedure.
- README containing setup, compilation, execution, and experiment instructions.
- Correctness verification.
- Benchmark results.
- MPI vs MapReduce comparison and analysis.
- Relevant plots/tables and observations.

For Q2 – gRPC, submit:

- Complete gRPC streaming system with multiple workers.
- .proto definition.
- Streaming client and CLI dashboard/query client.
- Dataset generator or reproducible dataset-generation procedure.
- README containing setup, compilation, execution, architecture, and experiment instructions.
- Correctness verification.
- Benchmark results.
- Relevant plots/tables and observations.

General Submission Requirements

The submission should be sufficiently documented for another person to reproduce the correctness tests and reported experiments. The README/report should clearly describe:

- The system architecture.
- Important implementation/design decisions.
- How the distributed workload is divided.
- How correctness was verified.
- How experiments were conducted.
- The main observations and conclusions.

There is intentionally no prescribed implementation strategy for the mapper/reducer decomposition or gRPC worker architecture. Students are expected to analyze the requirements and make appropriate design decisions.

Section 3: gRPC

Documentation

The following resources may be used to understand gRPC and Protocol Buffers:

- gRPC Documentation: <https://grpc.io/docs/>
- gRPC Introduction: <https://grpc.io/docs/what-is-grpc/introduction/>
- gRPC Core Concepts: <https://grpc.io/docs/what-is-grpc/core-concepts/>
- gRPC C++ Basics: <https://grpc.io/docs/languages/cpp/basics/>
- gRPC Python Basics: <https://grpc.io/docs/languages/python/basics/>
- Protocol Buffers: <https://protobuf.dev/overview/>

Problem 1: Collaborative Document Editing using gRPC

In this problem, you are required to build a multi-user collaborative document system using gRPC. The system consists of a central Document Server and multiple clients. Documents are maintained at the server, and multiple clients can access and modify the same document. When a client modifies a document, other clients currently subscribed to the same document should receive the updated document.

1.1 Document Service API

The following API methods should be implemented:

- `CreateDocument(CreateDocumentRequest) → CreateDocumentResponse`: Creates a new document with a unique name and initial content.
- `GetDocument(GetDocumentRequest) → GetDocumentResponse`: Returns the current contents of a document.
- `EditDocument(EditDocumentRequest) → EditDocumentResponse`: Inserts text at a specified position in an existing document.
- `SubscribeToUpdates(UpdateRequest) → stream DocumentUpdate`: Allows a client to subscribe to a document. Whenever the document is modified, the server sends the updated content to all subscribed clients.

1.2 Document Format

Each document consists of a unique name and text content. For example:

Document: `report.txt`

The `EditDocument` operation should support insertion of text at a specified character position. For example:

```
Document: Hello World
Position: 6
Text: "Distributed "
Result: Hello Distributed World
```

1.3 Components

1. gRPC Document Server The server should:

- Maintain documents in memory.
- Handle requests from multiple clients.
- Create and retrieve documents.
- Apply edits to documents.
- Maintain subscribers for each document.
- Send updates to subscribed clients whenever a document changes.
- Safely handle simultaneous edits from multiple clients.

The server address should be accepted as a command-line argument. Example:

```
python server.py localhost:50051
```

2. gRPC Client The client should provide a CLI with the following options:

1. Create Document
2. Open Document
3. Edit Document
4. Subscribe to Updates
5. Exit

A client should be able to receive document updates while continuing to interact with the server.

1.4 Task Breakdown

1. **Document Management:** Implement creation and retrieval of documents.
2. **Document Editing:** Implement insertion of text at a specified position.
3. **Concurrent Access:** Multiple clients may send edits to the same document simultaneously. The server must ensure that concurrent modifications do not corrupt the document.
4. **Update Propagation:** Whenever a document is modified, the server should send the latest document contents to all clients subscribed to that document.
5. **Streaming:** `SubscribeToUpdates` must use an appropriate gRPC server-streaming RPC.
6. **Synchronization:** Appropriate synchronization mechanisms should be used at the server to protect shared document state.

Note: Students are not required to implement Operational Transformation (OT) or CRDTs. If multiple edits arrive concurrently, the server may apply them in the order in which they are processed.

1.5 Example

Suppose the server is started as:

```
python server.py localhost:50051
```

Two clients connect to the server.

Step 1: Create Document Client 1 creates a document:

```
> create report.txt "Hello World"
[Client] Document report.txt created.
```

Step 2: Open Document Both clients open the document:

Client 1:

```
> open report.txt
[Client] Hello World
```

Client 2:

```
> open report.txt
[Client] Hello World
```

Step 3: Subscribe to Updates Client 2 subscribes to updates:

```
> subscribe report.txt
[Client] Subscribed to updates.
```

Step 4: Edit Document Client 1 performs:

```
> edit report.txt 6 "Distributed "
```

The server updates the document to `Hello Distributed World`. Client 2 should automatically receive:

```
[Update] Document report.txt modified.
Hello Distributed World
```

The update should be received without Client 2 explicitly requesting the document again.

Step 5: Concurrent Edits Suppose two clients send edits at approximately the same time:

- Client 1: Insert `"Distributed "` at position 6
- Client 2: Insert `"Systems "` at position 6

The server must safely process both requests without corrupting the document.

1.6 Deliverables

- **Proto File:** Define `DocumentService` and all required messages.
- **server.py:** Implements document storage, editing, synchronization, and update streaming.
- **client.py:** Implements the client CLI and gRPC communication.
- **README.md:** Documents how to run the system and demonstrates the system using multiple clients.

The demonstration should include:

1. Creating a document.
2. Two clients accessing the same document.
3. A client subscribing to updates.
4. One client modifying the document.
5. Another client receiving the update automatically.
6. Two clients attempting to modify the document concurrently.

Problem 2: Food Ordering System using gRPC

In this problem, you are required to build a simplified food ordering system using gRPC, similar to applications such as Swiggy. The system consists of a central Food Ordering Server, multiple Customer Clients, and multiple Restaurant Clients. Customers can view restaurants, place orders, and track their orders. Restaurants receive orders and update their status. The server maintains the state of restaurants and orders and coordinates communication between customers and restaurants.

2.1 Food Ordering Service API

The following API methods should be implemented:

- **ListRestaurants(RestaurantRequest) → RestaurantResponse:** Returns the list of available restaurants and their food items.
- **PlaceOrder(OrderRequest) → OrderResponse:** Allows a customer to place an order. The request should contain the restaurant name and selected food items. The server assigns a unique order ID.
- **GetOrderStatus(OrderStatusRequest) → OrderStatusResponse:** Returns the current status of an order.
- **UpdateOrderStatus(OrderStatusUpdate) → Acknowledge:** Allows a restaurant to update the status of an order.
- **SubscribeToOrderUpdates(OrderRequest) → stream OrderUpdate:** Allows a customer to subscribe to an order and receive status updates whenever the order changes.

2.2 Order Status

An order should move through the following states:

PLACED -> ACCEPTED -> PREPARING -> READY

An order may also be cancelled:

PLACED -> CANCELLED

The server should ensure that invalid state transitions are not allowed. For example, an order in the READY state cannot be changed back to PREPARING.

2.3 Components

1. gRPC Food Ordering Server The server should:

- Maintain a predefined list of restaurants and food items.
- Maintain the state of all orders.
- Generate a unique order ID for every order.
- Handle requests from multiple customers and restaurants.
- Validate order status transitions.
- Notify subscribed customers whenever their order status changes.
- Safely handle simultaneous requests.

The server address should be accepted as a command-line argument. Example:

```
python server.py localhost:50051
```

2. Customer Client The customer client should provide:

1. List Restaurants
2. Place Order
3. Check Order Status
4. Track Order
5. Cancel Order
6. Exit

3. Restaurant Client The restaurant client should provide:

1. View Pending Orders
2. Accept Order
3. Start Preparing
4. Mark Ready
5. Exit

2.4 Restaurant and Food Data

The server should maintain a predefined set of restaurants. For example:

```
Restaurant: Pizza House
  Margherita Pizza - 250
  Farmhouse Pizza  - 350
  Garlic Bread     - 150
```

```
Restaurant: Burger Point
  Veg Burger      - 180
  Cheese Burger   - 220
  French Fries    - 120
```

Students are not required to implement restaurant registration or menu management.

2.5 Task Breakdown

1. **Restaurant Listing:** Customers should be able to retrieve the available restaurants and their food items.
2. **Order Placement:** A customer selects a restaurant and one or more food items and submits an order. The server assigns a unique order ID and sets its initial status to PLACED.
3. **Order Processing:** The restaurant should be able to accept an order and subsequently change its status to PREPARING and READY.
4. **Order Tracking:** Customers should be able to query the current status of their order.
5. **Real-Time Updates:** Customers should be able to subscribe to an order and receive status updates using gRPC streaming.
6. **Concurrency:** Multiple customers and restaurants may send requests simultaneously. The server must safely maintain the state of each order.

2.6 Exception Handling

The server should handle the following situations using appropriate gRPC status codes:

- Customer attempts to order from a non-existent restaurant.
- Customer attempts to order an unavailable food item.
- Customer attempts to access a non-existent order.
- Customer attempts to cancel an order that has already been accepted or prepared.
- Restaurant attempts to update an order belonging to another restaurant.
- An invalid order status transition is requested.

The server should return an appropriate error instead of silently ignoring such requests.

2.7 Example

Suppose the server is started as:

```
python server.py localhost:50051
```

Step 1: Browse Restaurants The customer requests the available restaurants:

```
> restaurants
[Server]
1. Pizza House
   - Margherita Pizza : 250
   - Farmhouse Pizza  : 350
   - Garlic Bread     : 150
2. Burger Point
   - Veg Burger       : 180
   - Cheese Burger    : 220
```

Step 2: Place Order The customer places an order:

```
> order Pizza House "Margherita Pizza" 1 "Garlic Bread" 2
[Client] Order placed successfully.
[Client] Order ID: 0101
[Client] Total: 550
[Client] Status: PLACED
```

Step 3: Restaurant Processes Order The restaurant views its pending orders:

```
> pending
[Restaurant] Order 0101 : PLACED
```

The restaurant accepts and processes the order:

```
> accept 0101
[Restaurant] Order 0101 : ACCEPTED

> prepare 0101
[Restaurant] Order 0101 : PREPARING

> ready 0101
[Restaurant] Order 0101 : READY
```

Step 4: Customer Tracks Order The customer subscribes to the order:

```
> track 0101
[Client] Tracking order 0101...
```

Whenever the restaurant changes the order status, the customer receives:

```
[Update] Order 0101 : ACCEPTED
[Update] Order 0101 : PREPARING
[Update] Order 0101 : READY
```

The customer should receive these updates without explicitly polling the server.

Step 5: Exception Example Suppose the restaurant attempts an invalid transition:

```
> prepare 0101  
[Error] Invalid order state transition.
```

The server should return an appropriate gRPC error status.

2.8 Deliverables

- **Proto File:** Define `FoodOrderingService` and the required request, response, and streaming messages.
- **server.py:** Implements restaurant data, order management, concurrency handling, exception handling, and streaming updates.
- **customer.py:** Implements customer operations.
- **restaurant.py:** Implements restaurant operations.
- **README.md:** Documents how to run the system and demonstrates the system using multiple clients.

The demonstration should include:

1. Listing restaurants.
2. Placing an order.
3. A restaurant processing the order.
4. A customer receiving real-time order updates.
5. At least two clients interacting with the server concurrently.
6. At least two exceptional cases handled using gRPC status codes.